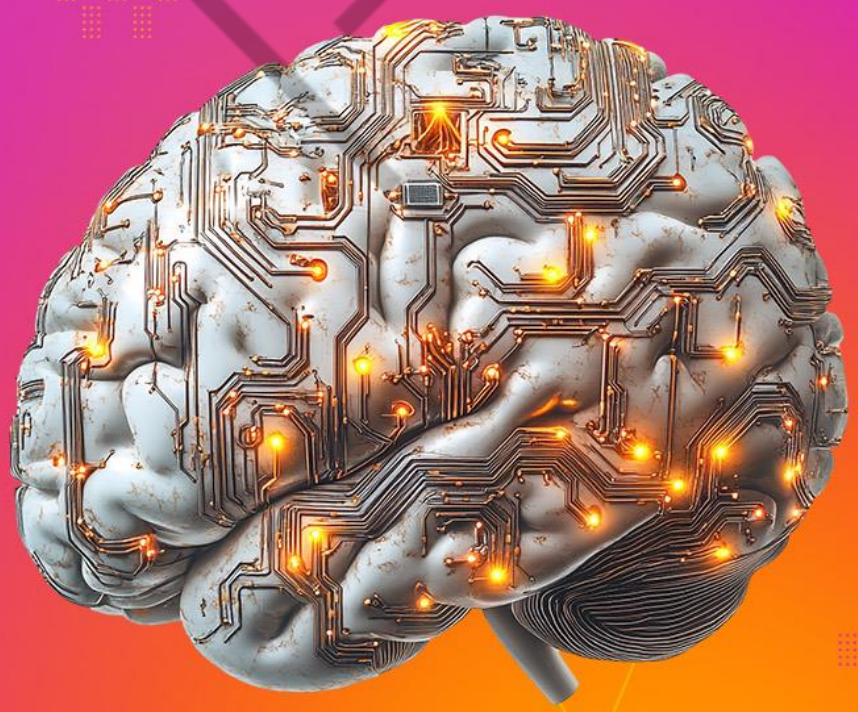
FIAP ON

TRATORES DIGITAIS - HARDWARES PARA IA

COMUNICAÇÃO SEM FIO: O FUTURO DA CONECTIVIDADE GLOBAL



7

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Circuito do liga/desliga LED usando botão e interrupção.....12
Figura 2 - Borda de subida em azul, borda de descida em verde 15
Figura 3 - Testando o Wi-Fi GUEST do Wokwi 21
Figura 4 - Limites do plano gratuito do Wokwi para uso do Wi-Fi 22
Figura 5 - ESP32 modelo DO IT.....22
Figura 6 - Documentação do Wokwi informando que o Bluetooth ainda não é suportado 28
Figura 7 - ESP32 modelo DOIT.....31

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo simples para entender o uso da interrupção no ESP32...7
Código-fonte 2 - Declaração de pinos e variáveis do código 7
Código-fonte 3 - Declaração da interrupção com parâmetros.....8
Código-fonte 4 - Declaração da interrupção propriamente dita.....8
Código-fonte 5 - Montagem do void setup() 9
Código-fonte 6 - Sintaxe da declaração de uma interrupção 10
Código-fonte 7 - Declaração de parâmetros obrigatórios à declaração uma interrupção 10
Código-fonte 8 - Void loop() vazio indicando que não precisa usar 10
Código-fonte 9 - Algoritmo liga/desliga LED melhorado com antiboucing no tempo do botão 13
Código-fonte 10 - Tipos de interrupções a partir do sinal externo.....14
Código-fonte 11 - Exemplo básico do uso do Wi-Fi no ESP32, exibindo o IP quando conectado.....20
Código-fonte 12 - Código de exemplo que usa Wi-Fi para medir a potência do sinal Wi-Fi.....25
Código-fonte 13 - Exemplo básico de uso do Bluetooth Clássico ligando e desligando um LED através do ESP32.....29
Código-fonte 14 - Exemplo básico de BLE para ligar/desligar um LED 33

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 COMUNICAÇÃO SEM FIO: O FUTURO DA CONECTIVIDADE GLOBAL 5
1.1 Introdução 5
1.1.1 Funcionamento da Interrupção.....6
1.1.2 Aplicações Reais de Interrupção.....15
2 WI-FI DO ESP32 18
2.1 Configurações Básicas do Wi-Fi 18
2.1.1 Usando Wokwi 20
2.1.2 Usando o ESP32 Real.....22
3 BLUETOOTH 26
3.1 Bluetooth Clássico.....27
3.2 Exemplo de Código Bluetooth Clássico 28
3.2.1 Explicação do Código.....29
3.2.2 Como Testar o Código? 30
3.3 Bluetooth Low Energy (BLE) 30
3.3.1 Código Controle de LED com BLE 31
3.3.2 Explicação do Código BLE 34
3.3.3 Como Testar.....34
3.3.4 Saída Esperada.....35
CONCLUSÃO.....36
REFERÊNCIAS.....38

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 COMUNICAÇÃO SEM FIO: O FUTURO DA CONECTIVIDADE GLOBAL

1.1 Introdução

As interrupções no ESP32 desempenham um papel diferente em sistemas de IA embarcada, especialmente em aplicações que exigem respostas rápidas dos sensores.

Elas permitem que o microcontrolador pause temporariamente o fluxo de execução do programa para atender a eventos externos ou internos de alta prioridade, como leituras de sensores ou sinais críticos.

Isso garante que nenhuma informação importante seja perdida, mesmo em cenários onde múltiplas tarefas estão sendo executadas simultaneamente.

No contexto da IA, as interrupções ajudam a capturar dados de sensores em tempo real. Por exemplo, em um sistema de monitoramento de saúde, os sinais vitais, como frequência cardíaca e temperatura, podem ser lidos por sensores conectados ao ESP32.

Utilizar interrupções para processar essas entradas em tempo real garante que os dados alimentem modelos de IA sem atraso ou inconsistência, aumentando a precisão das previsões.

Além disso, interrupções são úteis para otimizar o desempenho do ESP32 em aplicações complexas. Como o microcontrolador possui recursos limitados, a eficiência no uso de energia e processamento é importante. Vale mencionar que o uso de interrupção também evita desperdício de ciclos de CPU, permitindo que o ESP32 entre em modo de espera e apenas desperte quando necessário, como ao detectar um movimento com um sensor PIR em um sistema de vigilância inteligente, por exemplo.

Em sistemas de IA distribuídos, as interrupções facilitam a comunicação com outros dispositivos. Por exemplo, em uma rede de IoT, o ESP32 pode utilizar interrupções para responder a mensagens críticas de outros nós ou para sincronizar dados em tempo real.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Isso ajuda bastante as aplicações como agricultura inteligente, onde vários sensores e atuadores trabalham em conjunto para otimizar o uso de recursos naturais.

Assim, as interrupções tornam o ESP32 uma ferramenta altamente eficiente e responsiva para implementar soluções de IA embarcada. Elas não apenas melhoram o desempenho do sistema, mas também garantem confiabilidade e precisão, fatores básicos em aplicações que conectam a IA ao mundo físico.

1.1.1 Funcionamento da Interrupção

A interrupção é um recurso de qualquer ESP32. Ela vem nativa de fábrica, mas fica “paradinha” sem funcionar internamente até você ativá-la através de uma linha de código.

Do ponto de vista do hardware, uma **interrupção no ESP32** é um mecanismo que permite ao microcontrolador responder a eventos externos ou internos imediatamente, pausando a execução normal do programa que está sob o comando do **void setup()** ou **void loop()** para executar uma tarefa de alta prioridade.

Esse mecanismo é fundamental para lidar com sinais ou condições que exigem uma resposta rápida, sem precisar que o programa principal **void loop()** fique constantemente verificando esses eventos (técnica conhecida como **polling**).

Após a devida configuração no código-fonte, uma interrupção é solicitada por uma **fonte de interrupção**, que geralmente é um evento externo, como uma mudança de estado em um pino GPIO.

Quando a interrupção ocorre, o hardware do ESP32 salva a posição atual do programa e desvia o fluxo de execução para uma função chamada **rotina de serviço de interrupção** (ISR — Interrupt Service Routine), onde a tarefa associada ao evento é executada. Vamos a um código para você entender isso melhor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#define BUTTON_PIN 18  // Pino do botão
#define LED_PIN 2      // Pino do LED

volatile bool ledState = false;  // Variável para armazenar o
estado do LED (volátil para interrupção)

// Função de serviço de interrupção (ISR)
void IRAM_ATTR botaoPressionado() {
  ledState = !ledState;  // Alterna o estado do LED
  digitalWrite(LED_PIN, ledState);  // Atualiza o LED
}

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // Configura o botão como
entrada com resistor pull-up interno
  pinMode(LED_PIN, OUTPUT);           // Configura o LED como
saída
  digitalWrite(LED_PIN, LOW);        // Garante que o LED começa
apagado

  // Anexa a interrupção ao botão (acionada na borda de descida)
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN),
botaoPresssionado, FALLING);

  Serial.begin(115200);  // Inicializa a comunicação serial para
depuração
  Serial.println("Sistema  iniciado.  Pressione  o  botão  para
alternar o LED.");
}

void loop() {
  // O loop principal pode ficar vazio ou realizar outras tarefas
}
```

Código-fonte 1 - Exemplo simples para entender o uso da interrupção no ESP32
Fonte: Elaborado pelo autor (2024)

Vamos à explicação didática do código. Primeiro, sempre fazemos a declaração de pinos e variáveis.

```
#define BUTTON_PIN 18  // Pino conectado ao botão
#define LED_PIN 2      // Pino conectado ao LED

volatile bool ledState = false;  // Variável para armazenar o
estado do LED
```

Código-fonte 2 - Declaração de pinos e variáveis do código
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **BUTTON_PIN (18):** define o GPIO 18 como o pino onde o botão está conectado. Note que essa definição BUTTON_PIN vai junto com a declaração da interrupção que é "botaoPressionado". Calma que vamos explorar isso mais aqui.

```
attachInterrupt(digitalPinToInterrupt(BUTTON_PIN),
botaoPresssionado, FALLING);
```

Código-fonte 3 - Declaração da interrupção com parâmetros
Fonte: Elaborado pelo autor (2024)

- **LED_PIN (2)** define o GPIO 2 como o pino onde o LED está conectado.
- **volatile bool ledState** armazena o estado atual do LED (ligado ou desligado). É marcada como *volatile* porque pode ser alterada tanto pelo programa principal quanto pela **ISR** (função de interrupção). Esse **volatile** é novidade aqui no curso, então vamos entender o porquê! A explicação está no comportamento do compilador. Durante a otimização do código que será gravado no ESP32, o compilador pode assumir que **o valor de uma variável** não muda se não houver nada no código principal que a altere — isto é, nada citado no *void loop()* ou *void setup()*. Essa suposição pode ocasionar comportamentos imprevisíveis em situações onde o valor da variável é alterado por uma **ISR (Interrupt Service Routine)** ou por hardware. Ao declarar uma variável como **volatile**, você instrui o compilador **sempre ler o valor real da variável**, isto é, o compilador não armazenará o valor em um registro da CPU ou fará cache, e sim, ele sempre acessará a memória real onde a variável está armazenada. Além disso, o **volative** impede que o compilador remova ou reorganize instruções que acessam a variável.

Seguindo com a próxima parte do código.

```
void IRAM_ATTR botaoPressionado() {
  ledState = !ledState;  // Alterna o estado do LED
  digitalWrite(LED_PIN, ledState);  // Atualiza o LED
}
```

Código-fonte 4 - Declaração da interrupção propriamente dita
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **IRAM_ATTR** garante que a ISR seja armazenada na RAM (em vez da flash), o que a torna mais rápida.
- **botaoPressionado** é a rotina de serviço de interrupção (ISR) chamada quando o botão é pressionado. Ela alterna o estado da variável *ledState* (de **false** para **true** ou vice-versa), e atualiza o LED pelo digitalWrite que liga ou desliga o LED com true ou false. Portanto, esse nome, “botaoPressionado”, é uma função que deverá estar atrelada à declaração da interrupção. Quando a interrupção é executada, isto é, quando as linhas abaixo do void IRAM ATTR botaoPressionado() são interpretadas, o ESP32 retorna de onde saiu quando o botão foi pressionado.

```
void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // Configura o botão como
entrada com resistor pull-up interno
  pinMode(LED_PIN, OUTPUT);            // Configura o LED como
saída
  digitalWrite(LED_PIN, LOW);         // Garante que o LED começa
apagado

  // Anexa a interrupção ao botão (acionada na borda de descida)
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN),
handleButtonPress, FALLING);

  Serial.begin(115200);  // Inicializa a comunicação serial para
depuração
  Serial.println("Sistema iniciado. Pressione o botão para
alternar o LED.");
}
```

Código-fonte 5 - Montagem do void setup()
Fonte: Elaborado pelo autor (2024)

- **Configuração do botão (pinMode(BUTTON_PIN, INPUT_PULLUP))**, linha que define o botão como entrada e ativa o resistor pull-up interno. O estado padrão do pino é **HIGH** (3.3V) e vai para **LOW** (0V) quando o botão é pressionado. Observe que o desenho do circuito tem um botão conectado ao negativo do circuito (GND). Portanto, a entrada GPIO 18 receberá GND (0V) quando o botão for pressionado manualmente pelo usuário. E quando não estiver pressionando, o GPIO 18 recebe VCC (3,3V) internamente, via um resistor nativo da arquitetura do ESP32 ativado pelo comando de programação **INPUT_PULLUP** dentro do pinMode.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Configuração do LED (pinMode(LED_PIN, OUTPUT))**, na qual define o LED como saída para que seu estado possa ser controlado pelo programa.
- **attachInterrupt**, onde temos:
  - Configuração de uma interrupção atrelando ao **BUTTON_PIN**.
  - Acionamento da ISR botaoPressionado() quando ocorre uma **borda de descida** (FALLING), ou seja, quando o botão é pressionado, o sinal da tensão sai do VCC 3,3V (via resistor interno habilitado) e cai para o GND (0V), isto é, a tensão caiu. Portanto, voltando à declaração inteira da interrupção, você tem os seguintes parâmetros:

```
attachInterrupt(digitalPinToInterrupt(PINO BOTAO), NOME DA
FUNÇÃO, TIPO DE BORDA DO SINAL DO BOTÃO);
```

Código-fonte 6 - Sintaxe da declaração de uma interrupção
Fonte: Elaborado pelo autor (2024)

Comparando a sintaxe da declaração da interrupção com o código exemplo, podemos entender totalmente agora:

```
attachInterrupt(digitalPinToInterrupt(BUTTON_PIN),
botaoPresssionado, FALLING);
```

Código-fonte 7 - Declaração de parâmetros obrigatórios à declaração uma interrupção
Fonte: Elaborado pelo autor (2024)

- **Serial Monitor** exibe mensagens para depuração e confirma que o sistema está funcionando.

Para finalizar, o *void loop()* está vazio porque não precisamos monitorar o botão, pois a ideia da interrupção é justamente ela monitorar sozinha, e quando BUTTON_PIN alterar seu nível de tensão no sentido de queda (FALLING) de 3,3V para GND, a interrupção acorda e se executa.

```
void loop() {
  // O loop principal pode ficar vazio ou realizar outras tarefas
}
```

Código-fonte 8 - Void loop() vazio indicando que não precisa usar
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- O *loop()* é deixado vazio porque o controle do LED é feito exclusivamente pela interrupção. Isso mostra como o uso de interrupções pode “liberar” o programa principal para outras tarefas ou mantê-lo simples.

Portanto, resumindo esse código, temos:

1) **Estado inicial:**
   - O LED começa apagado (estado LOW).
2) **Pressionando o botão:**
   - Quando o botão é pressionado, ocorre uma **borda de descida** no GPIO 18.
   - Isso aciona a interrupção, chamando a função botaoPressionado.
3) **Atualização do LED:**
   - A ISR alterna o estado de *ledState* (de **false** para **true** ou **vice-versa**) e liga/desliga o LED no GPIO 2.
4) **Repetição:**
   - O LED muda de estado a cada vez que o botão é pressionado.

Segue o circuito para você montar no seu Wokwi.com:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

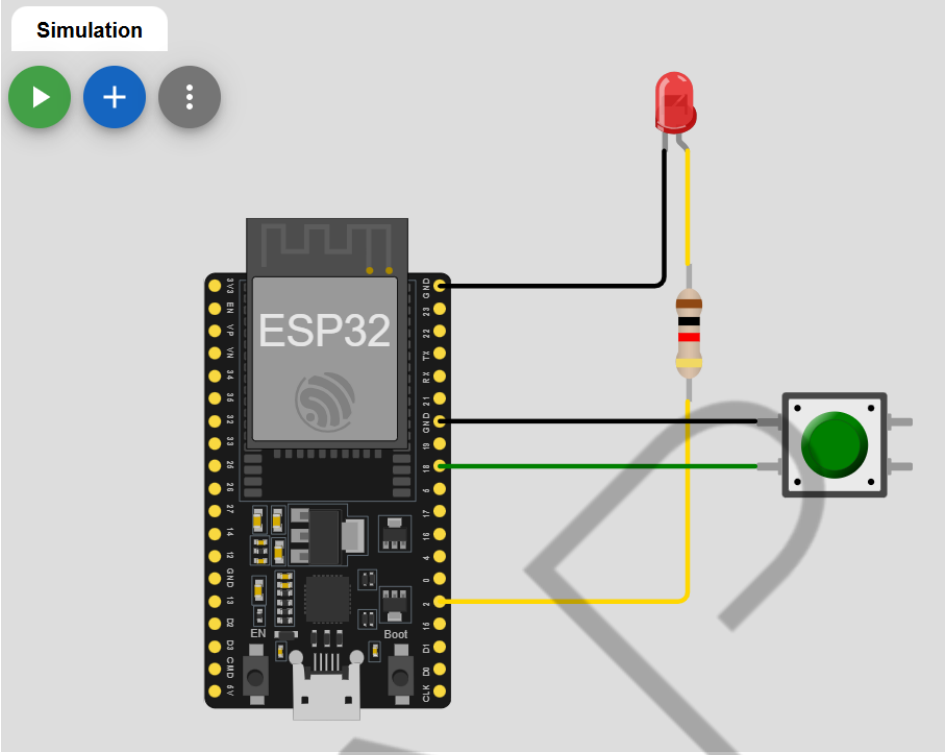


Figura 1 - Circuito do liga/desliga LED usando botão e interrupção
Fonte: Elaborado pelo autor (2024)

Você vai notar que ao apertar o botão, o LED pode piscar e terminar desligado. Isso acontece porque ao pressionar o botão, o programa é tão rápido que pode ler o botão mais de uma vez e repetir o comando da interrupção, fazendo o LED desligar e ligar apenas com uma pressionada “lenta”. Se pressionar mais rápido, esse efeito não acontece. Experimente!

Agora, podemos adicionar uma melhoria neste código removendo esse efeito no controle do tempo do botão pressionado. Veja essa versão melhorada do código e compare com o anterior.

A melhoria está no acréscimo do *millis()* dentro da função da interrupção, onde ele “atrasa” o liga/desliga do LED de acordo com o valor da variável *debounceDelay*, evitando assim, uma nova chamada de interrupção durante a mesma clicada no botão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#define BUTTON_PIN 18  // Pino conectado ao botão
#define LED_PIN 2      // Pino conectado ao LED

volatile bool ledState = false;  // Variável para armazenar o
estado do LED
volatile unsigned long lastDebounceTime = 0;   // Armazena o
último tempo de acionamento
const unsigned long debounceDelay = 500;  // Tempo de debounce
em milissegundos

// Função de serviço de interrupção (ISR)
void IRAM_ATTR botaoPressionado() {
  unsigned long currentTime = millis();   // Tempo atual em
milissegundos

  // Verifica se o tempo decorrido é maior que o tempo de
debounce
  if ((currentTime - lastDebounceTime) > debounceDelay) {
    ledState = !ledState;  // Alterna o estado do LED
    digitalWrite(LED_PIN, ledState);  // Atualiza o LED
    lastDebounceTime = currentTime;  // Atualiza o último tempo
de acionamento
  }
}

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // Configura o botão como
entrada com resistor pull-up interno
  pinMode(LED_PIN, OUTPUT);            // Configura o LED como
saída
  digitalWrite(LED_PIN, LOW);        // Garante que o LED começa
apagado

  // Anexa a interrupção ao botão (acionada na borda de descida)
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN),
botaoPressionado, FALLING);

  Serial.begin(115200);  // Inicializa a comunicação serial para
depuração
  Serial.println("Sistema iniciado. Pressione o botão para
alternar o LED.");
}

void loop() {
  // O loop principal pode ficar vazio ou realizar outras tarefas
}
```

Código-fonte 9 - Algoritmo liga/desliga LED melhorado com antiboucing no tempo do botão
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Retomando a sintaxe da interrupção, observe o último parâmetro repassado (FALLING). O ESP32 suporta diferentes tipos de passagem nesse parâmetro, como:

- **Interrupções de subida:** ativadas por uma mudança no estado do pino (subida de 0V (GND) para 3,3V (VCC) → RISING.
- **Interrupções de mudança de nível:** disparadas quando o estado do pino altera seu nível, independente se está subindo ou descendo → CHANGE.
- **Interrupções de descida:** como já vista no código e que vamos repetir aqui a título de comparação.

Considerando o exemplo apresentado, essas diferentes interrupções ficariam assim:

```
attachInterrupt(digitalPinToInterrupt(pino),funcaoISR, RISING);
// Ativada na borda de subida
attachInterrupt(digitalPinToInterrupt(pino),funcaoISR, CHANGE);
// Ativada em qualquer mudança de estado
attachInterrupt(digitalPinToInterrupt(pino),funcaoISR,
FALLING);  // Ativada na borda de descida
```

Código-fonte 10 - Tipos de interrupções a partir do sinal externo
Fonte: Elaborado pelo autor (2024)

Na figura a seguir, é ilustrado um gráfico que representa os tipos de excitação da interrupção. Essa borda de subida ou descida é do sinal ligado ao botão de pressão, que pode estar chaveando um sinal positivo ou negativo. E enquanto ele não chaveia, o sinal de entrada deve ficar fixo no respectivo nível oposto, isto é, se o botão chaveia o positivo ao ser pressionado, então, quando não-pressionado, entra o negativo e vice-versa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

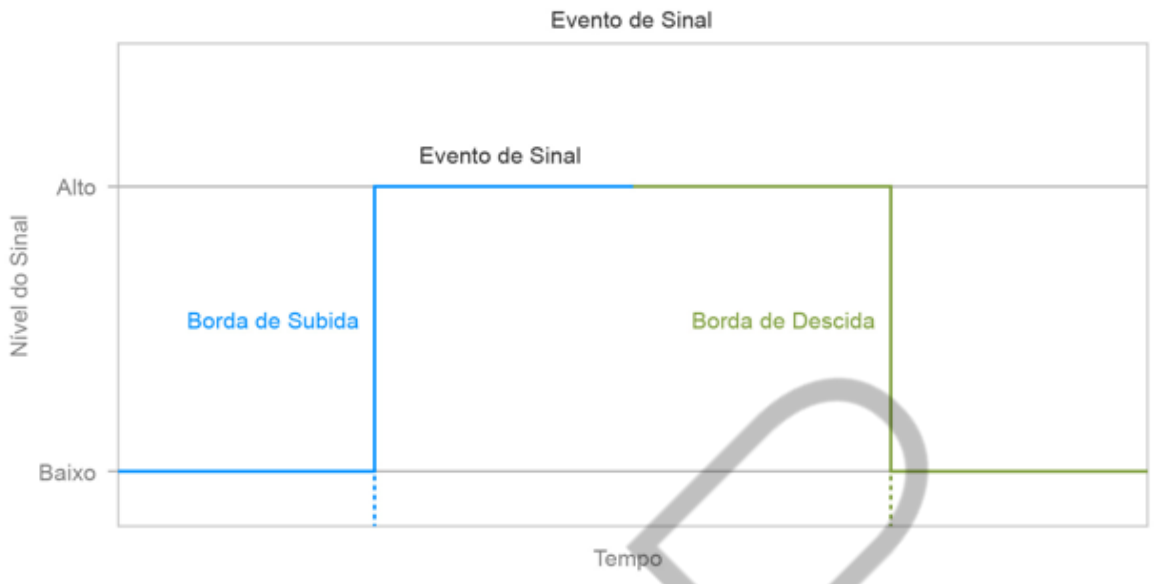


Figura 2 - Borda de subida em azul, borda de descida em verde
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

1.1.2 Aplicações Reais de Interrupção

Uma aplicação prática que usa interrupções no ESP32 é o monitoramento em tempo real de eventos rápidos, como o número de pessoas que atravessam uma catraca ou a contagem de giros de uma peça rotativa em uma máquina.

Ambas as situações envolvem eventos que podem ocorrer de forma intermitente e, muitas vezes, muito rápido para serem monitorados com eficiência usando o método tradicional de verificação contínua (*polling*) com if-else. Com interrupções, o ESP32 pode capturar cada evento de maneira instantânea e precisa, sem desperdício de recursos.

Imagine o cenário de uma catraca em um sistema de controle de acesso. Um sensor infravermelho (IR) detecta a passagem de uma pessoa. Cada vez que o sensor detecta um objeto (como o corpo de uma pessoa) interrompendo o feixe de luz, ele gera um sinal de mudança no nível lógico (de HIGH para LOW ou vice-versa). Esse sinal é conectado a um pino do ESP32 configurado para disparar uma interrupção. Quando a interrupção é ativada, o ESP32 pausa brevemente o que está fazendo, executa uma rotina especial (ISR) para registrar o evento, como incrementar uma variável *x*++ que representa o número total de pessoas que passaram por ali.

Agora, considere um cenário industrial, onde um sensor de efeito Hall está instalado em um motor rotativo (sensor de ímã que sente a passagem do campo

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

magnético de um ímã móvel preso ao eixo girante da máquina). Esse sensor gera pulsos toda vez que um ímã fixado na peça giratória passa pelo sensor.

A cada pulso, uma interrupção no ESP32 é acionada, permitindo contar o número de giros. Combinando a contagem de pulsos com um temporizador, é possível calcular a velocidade de rotação (em rotações por minuto, RPM). Isso é extremamente útil em monitoramento de máquinas para detecção de problemas ou controle de velocidade.

O uso de interrupções torna o ESP32 muito eficiente, pois o microcontrolador não precisa verificar constantemente o estado do sensor. Ele pode realizar outras tarefas, como exibir os dados em um display ou se comunicar com um servidor remoto, enquanto garante que cada evento de passagem ou giro seja registrado com precisão.

Essa abordagem também reduz o consumo de energia, pois o ESP32 pode ficar em estado de baixa potência até que uma interrupção ocorra. Assim, interrupções são ideais para aplicações que exigem reatividade e eficiência em ambientes com eventos rápidos ou imprevisíveis.

Na área hospitalar, uma aplicação semelhante que utiliza interrupções no ESP32 é o monitoramento de fluxo de pessoas em áreas restritas, como salas de cirurgia ou Unidades de Terapia Intensiva (UTIs), e o controle de equipamentos médicos que operam em ciclos, como bombas de infusão. A contagem precisa de eventos e a resposta rápida são cruciais nesse tipo de ambiente, e as interrupções oferecem a confiabilidade necessária.

Por exemplo, em um hospital, podemos instalar um sensor infravermelho em uma porta de acesso à UTI para monitorar o número de profissionais que entram e saem da área restrita. Cada vez que o feixe de luz do sensor é interrompido, uma interrupção é acionada no ESP32, que registra a entrada ou saída de uma pessoa.

O sistema pode distinguir entre entrada e saída com dois sensores instalados em sequência (criando uma “zona de passagem”). Esse tipo de monitoramento é básico para garantir que as condições de segurança sejam mantidas, como a limitação do número de pessoas dentro da UTI para evitar contaminação.

Outro exemplo é o controle de ciclos de uma bomba de infusão usada para administrar medicamentos intravenosos. Sensores conectados ao ESP32 podem

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

monitorar a rotação de uma peça no mecanismo da bomba que controla o fluxo de líquido.

Cada giro da peça pode ser detectado por um sensor de efeito Hall, acionando uma interrupção para registrar o número de giros, que corresponde diretamente ao volume de medicamento infundido. Esse monitoramento em tempo real garante que a dosagem administrada seja precisa e pode acionar alarmes em caso de irregularidades, como paradas inesperadas ou variação na velocidade.

Assim como no caso da catraca ou da contagem de giros industriais, as interrupções no algoritmo para o ambiente de hospital permitem que o ESP32 registre eventos críticos de forma confiável e em tempo real, liberando o microcontrolador para executar outras funções, como comunicar dados ao sistema central do hospital ou acionar alarmes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 WI-FI DO ESP32

O ESP32 é um microcontrolador poderoso e amplamente utilizado em aplicações de IoT devido à sua conectividade Wi-Fi integrada.

Ele suporta o padrão **Wi-Fi 802.11 b/g/n**, permitindo que dispositivos se conectem a redes sem fio para troca de dados. Esse recurso o torna atraente para criar sistemas inteligentes que exigem comunicação com a Internet, como automação residencial, monitoramento remoto e dispositivos conectados em nuvens IoT, aliados com sistemas de IA.

Uma das características fundamentais do Wi-Fi no ESP32 é sua capacidade de operar em diferentes modos. No modo **Station (STA)**, o ESP32 se comporta como um cliente que se conecta a um roteador Wi-Fi normal (como você agora, conectado a um roteador doméstico/empresarial, caso não esteja usando a conexão via celular), permitindo acesso à Internet ou a um servidor local. Esse é o modo mais comum em aplicações IoT, como envio de dados de sensores para servidores na nuvem.

No modo **Access Point (AP)**, o ESP32 cria sua própria rede Wi-Fi, à qual outros dispositivos podem se conectar diretamente. Esse modo é útil em cenários onde não há um roteador disponível, como para configurar dispositivos em campo ou para comunicação ponto a ponto. Esse modo também abre portas para criar soluções maliciosas no mundo da cibersegurança, que será visto futuramente.

Além disso, o ESP32 oferece o modo **AP+STA**, que combina as funcionalidades de cliente e ponto de acesso simultaneamente. Esse modo é particularmente interessante em aplicações que exigem controle local e remoto ao mesmo tempo. Por exemplo, o ESP32 pode atuar como cliente de um servidor na nuvem enquanto também disponibiliza uma rede Wi-Fi local para acesso direto por dispositivos próximos.

2.1 Configurações Básicas do Wi-Fi

Como já mencionado, o modo mais comum de configuração é o **Station Mode (STA)**, onde o ESP32 atua como um cliente, conectando-se a um roteador Wi-Fi.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para isso, é necessário fornecer o **SSID** (nome da rede) e a **senha**, permitindo que o dispositivo se autentique na rede e obtenha acesso à Internet. Esse modo é praticamente uma regra em aplicações IoT, como envio de dados de sensores para a nuvem ou controle remoto de dispositivos.

Com comandos simples e o uso de bibliotecas como WiFi.h, é possível configurar essas conexões de forma eficiente, preparando o ESP32 para operar em cenários reais de IoT.

Vejamos alguns exemplos em código:

```
#include <WiFi.h>  // Biblioteca para Wi-Fi

// Configurações da Rede Wi-Fi
const char* ssid = "NomeDaRede";       // Substitua pelo SSID da
sua rede Wi-Fi
const char* password = "SenhaDaRede"; // Substitua pela senha da
sua rede Wi-Fi

void setup() {
  Serial.begin(115200);  // Inicializa o Monitor Serial
  delay(1000);

  // Exibe uma mensagem inicial
  Serial.println("Conectando ao Wi-Fi...");

  // Inicia a conexão ao Wi-Fi no modo Station
  WiFi.begin(ssid, password);

  // Aguarda até conectar ou exibe mensagem de erro após timeout
  int maxAttempts = 20;  // Número máximo de tentativas
  int attempt = 0;
  while (WiFi.status() != WL_CONNECTED && attempt < maxAttempts)
{
    delay(500);
    Serial.print(".");
    attempt++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    // Conexão bem-sucedida
    Serial.println("\nWi-Fi conectado!");
    Serial.print("Endereço IP: ");
    Serial.println(WiFi.localIP());   //  Exibe  o  endereço  IP
atribuído ao ESP32
  } else {
    // Falha ao conectar
    Serial.println("\nFalha ao conectar ao Wi-Fi.");
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  }
}

void loop() {
  // O loop principal pode ser usado para outras tarefas
}
```

Código-fonte 11 - Exemplo básico do uso do Wi-Fi no ESP32, exibindo o IP quando conectado
Fonte: Elaborado pelo autor (2024)

O que o código faz? Ele utiliza o comando *WiFi.begin(ssid, password*) para iniciar a conexão com a rede. E então, o *loop* verifica o estado da conexão usando *WiFi.status()* até que o ESP32 esteja conectado ou que o número máximo de tentativas seja alcançado.

O Monitor Serial exibe mensagens indicando o progresso da conexão. Após a conexão, exibe o endereço IP atribuído ao ESP32.

Se o ESP32 não se conectar dentro de 10 segundos (20 tentativas de 500 ms), o código exibe uma mensagem de falha.

Como testar? Existem duas formas:

2.1.1 Usando Wokwi

Substitua *NomeDaRede* e *SenhaDaRede* pelo SSID “Wokwi-GUEST” e deixe a senha Wi-Fi vazia, apenas com “” sem espaços entre elas. A figura a seguir mostra isso. Com esse teste, você conseguirá **simular** uma rede Wi-Fi no Wokwi e vai observar um IP impresso no Monitor Serial, que nesse caso foi 10.10.0.2. Contudo, essa rede Wi-Fi não é real.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 3 - Testando o Wi-Fi GUEST do Wokwi
Fonte: Elaborado pelo autor (2024)

O plano gratuito do Wokwi não permite você usar um roteador privado de verdade aí no seu local (que eles chamam de Gateway Privado), que seria o SSID e a senha do seu roteador.

Importante: não incentivamos gastar recursos financeiros em plataformas de estudo para além do que já é investido na plataforma da FIAP. Então fica a critério do aluno querer fazer qualquer tipo de upgrade no Wokwi.com, sabendo que a rede Wokwi-GUEST atende bem. A documentação do Wokwi esclarece todas as limitações da conta gratuita, conforme a figura a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Gateway Publico	Gateway Privado
Velocidade	Mais devagar	Mais rápido
Estabilidade	Média	Alta
Localização	Remoto, na nuvem	Funciona no seu computador
Privacidade	Conexões monitoradas	Sem monitoramento
Conexões de saída	✅	✅
Conexões de entrada	❌	✅
Disponibilidade	Todos os usuários	Usuários pagantes

Figura 4 - Limites do plano gratuito do Wokwi para uso do Wi-Fi
Fonte: Wokwi (2024)

Portanto, o único jeito gratuito no Wokwi é a simulação de um sinal Wi-Fi. Contudo, existe uma segunda alternativa e melhor.

2.1.2 Usando o ESP32 Real

A segunda forma de testar o WiFi.h de um ESP32 é adquirindo um ESP32 de verdade, como mostra a figura “ESP32 modelo DO IT”.

Figura 5 - ESP32 modelo DO IT
Fonte: Manual.plus (2021)

Para funcionar a placa de verdade, você precisa instalar a biblioteca que contém o arquivo WiFi.h. O jeito mais fácil para você ter acesso à biblioteca WiFi.h no seu computador é utilizar a **IDE Arduino** configurada corretamente para trabalhar com

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

placas ESP32. Como já dito, a biblioteca WiFi.h já vem inclusa no **pacote de suporte ao ESP32**, e o processo é simples:

1) **Instale a IDE Arduino**, baixando e instalando a **IDE Arduino** a partir do site oficial: <https://www.arduino.cc/en/software>.
2) **Adicione à sua IDE, suporte ao ESP32**, abrindo a IDE Arduino e vá em Arquivo > Preferências. No campo "URLs adicionais para Gerenciadores de Placas", adicione o link: <https://dl.espressif.com/dl/package_esp32_index.json>, e em seguida clique em "OK".
3) **Instale o pacote do ESP32**, indo em Ferramentas > Placas > Gerenciador de Placas. Na barra de pesquisa, digite “esp32”. Clique em **Instalar** no pacote **ESP32 by Espressif Systems**. Aguarde a instalação do pacote. Tenha paciência pois demora alguns minutos mesmo.
4) Após a instalação do pacote, **selecione a Placa ESP32**, indo em Ferramentas > Placas. Selecione sua placa ESP32, como **"ESP32 Dev Module"** ou **"DOIT ESP32"**.
5) **Teste um exemplo com WiFi.h**, abrindo a IDE Arduino e crie um novo sketch. Copie e cole o exemplo básico fornecido anteriormente. Certifique-se de que o código inclui a linha: #include <WiFi.h>.
6) **Verifique a conexão e faça o upload**, para isso, conecte o ESP32 ao computador via cabo USB, selecione a porta correta em Ferramentas > Porta, e depois, clique no botão **Upload** para enviar o código para o ESP32.

Por que este é o método mais fácil de testar o WiFi? Porque a biblioteca WiFi.h é instalada automaticamente junto com o pacote ESP32. Não é necessário baixar ou configurar bibliotecas adicionais. A IDE Arduino é intuitiva, especialmente para iniciantes.

Se optar por essa segunda opção de teste, tenha certeza que o seu roteador é compatível com a frequência de 2,4GHz, e não somente 5GHz, pois é o que acontece com roteador muito moderno.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O ESP32 é projetado para operar exclusivamente em redes **Wi-Fi de 2,4 GHz**. Portanto, ele **não suporta redes de 5GHz**, um padrão mais recente que oferece maior velocidade em algumas aplicações, mas com menor alcance.

A frequência de **2,4GHz** é amplamente utilizada em dispositivos IoT, como o ESP32, devido a sua capacidade de oferecer **maior alcance** e melhor penetração através de obstáculos, como paredes e móveis. Essa frequência também é suportada pela maioria dos roteadores Wi-Fi modernos, o que a torna ideal para dispositivos embarcados e aplicações em ambientes domésticos ou industriais.

Essa limitação para 2,4GHz **não é um problema** para a maioria dos projetos de IoT, já que a maioria das redes Wi-Fi oferece suporte a ambas as bandas, 2,4GHz e 5GHz (redes de banda dupla).

O código-fonte para esse teste real é o mesmo apresentado no código-fonte “Exemplo básico do uso do Wi-Fi no ESP32, exibindo IP quando conectado”. Basta adicionar as credenciais do seu roteador local (usuário e senha) e ter certeza de que ele propaga sinal Wi-Fi em 2,4GHz.

Dica: você pode rotear o sinal Wi-Fi do seu celular para seu ESP32, desde que não seja o iPhone, porque a maioria desses celulares não liberam sinal 2,4GHz.

O código a seguir também apresenta um exemplo clássico do uso de Wi-Fi no ESP32, e um pouco mais aprimorado que o exemplo anterior. Esse código mede a potência do sinal Wi-Fi que sai do seu roteador e chega no microcontrolador. Com esse exemplo, você pode fazer um teste de qualidade do sinal WiFi na sua rede local sem fio. Segue faixas que indicam a qualidade da conexão:

- **-30 dBm:** sinal excelente.
- **-50 dBm a -70 dBm:** sinal bom.
- **-70 dBm a -90 dBm:** sinal fraco.
- **Abaixo de -90 dBm:** sinal muito ruim, conexão instável.

O valor de RSSI é exibido continuamente no Monitor Serial, atualizado a cada 2 segundos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#include <WiFi.h>

// Configurações da Rede Wi-Fi
const char* ssid = "seu SSID";  // Substitua pelo SSID da rede
Wi-Fi
const char* password = "SUA SENHA";          // Substitua pela
senha da rede (senha vazia no Wokwi-GUEST)

void setup() {
  Serial.begin(115200);  // Inicializa o Monitor Serial
  delay(1000);

  Serial.println("Conectando ao Wi-Fi...");

  // Inicia a conexão ao Wi-Fi
  WiFi.begin(ssid, password);

  // Aguarda a conexão
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  // Exibe informações da conexão
  Serial.println("\nWi-Fi conectado!");
  Serial.print("Endereço IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  // Obtém a intensidade do sinal (RSSI) em dBm
  int rssi = WiFi.RSSI();  // Lê a força do sinal da rede atual

  // Exibe a intensidade no Monitor Serial
  Serial.print("Intensidade do sinal Wi-Fi (RSSI): ");
  Serial.print(rssi);
  Serial.println(" dBm");

  delay(2000);  // Atualiza a leitura a cada 2 segundos
}
```

Código-fonte 12 - Código de exemplo que usa Wi-Fi para medir a potência do sinal Wi-Fi
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 BLUETOOTH

O ESP32 possui comunicação Bluetooth nativa, um dos recursos mais versáteis desse microcontrolador, permitindo comunicação sem fio eficiente em uma ampla variedade de aplicações, especialmente, com aplicativos de celular.

O ESP32 possui suporte a dois tipos principais de Bluetooth: o **Bluetooth Clássico** e o **Bluetooth Low Energy (BLE)**, cada um com características e finalidades específicas. O **Bluetooth Clássico** é projetado para situações em que é necessária a **transmissão contínua de dados**, como em sistemas de áudio, envio de arquivos ou comunicação serial entre dispositivos. Nesse modo, o ESP32 pode atuar como mestre ou escravo, possibilitando o envio de informações em tempo real com maior largura de banda.

Já o **Bluetooth Low Energy** foi projetado para aplicações que exigem **baixo consumo de energia**, o que o torna ideal para dispositivos que precisam operar por longos períodos com baterias pequenas, como sensores, relógios inteligentes e outros dispositivos vestíveis (*wearables*). O BLE trabalha de forma mais eficiente ao manter a comunicação em intervalos específicos, transmitindo pequenos pacotes de dados de forma otimizada. Esse modelo é amplamente utilizado em aplicações de **IoT**, onde o ESP32 coleta dados de sensores e os envia para outro dispositivo ou plataforma.

A presença simultânea dessas duas tecnologias no ESP32 oferece ao desenvolvedor uma flexibilidade interessante, permitindo adaptar a solução conforme as necessidades do projeto.

Por exemplo, o Bluetooth Clássico pode ser utilizado para criar uma comunicação serial simples entre o ESP32 e um computador ou um smartphone, enquanto o BLE pode ser aplicado em um projeto que monitora o nível de umidade do solo e envia atualizações ocasionais para um aplicativo móvel sem comprometer a autonomia do sistema.

Com esses dois modos integrados no ESP32, é possível conectar o microcontrolador tanto a dispositivos modernos, que utilizam BLE, quanto a sistemas mais tradicionais baseados em Bluetooth Clássico.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por esse motivo, o ESP32 é uma ferramenta poderosa para soluções de conectividade, equilibrando **eficiência energética**, **alcance** e **volume de dados**, características essenciais para projetos de IoT, automação e dispositivos conectados.

3.1 Bluetooth Clássico

A configuração do **Bluetooth Clássico** no ESP32 é uma forma prática e eficiente de realizar comunicação sem fio entre dispositivos, simulando uma comunicação **serial** tradicional.

O Bluetooth Clássico no ESP32 é suportado pela biblioteca **BluetoothSerial.h**, que torna o processo de configuração simples e intuitivo. Como visto anteriormente, o Clássico é amplamente utilizado em aplicações onde é necessário enviar e receber dados de forma contínua, como em sistemas de monitoramento, controle de dispositivos ou transmissão de mensagens.

Nessa comunicação, o ESP32 pode atuar tanto como **mestre**, iniciando a conexão com outro dispositivo, quanto como **escravo**, esperando que outro dispositivo (como um smartphone ou um computador) inicie a conexão.

Um exemplo prático é utilizar o ESP32 para enviar e receber mensagens de um aplicativo de terminal Bluetooth em um smartphone. Nesse cenário, o ESP32 atua como **escravo** e aguarda uma conexão. Uma vez conectado, ele pode receber comandos para ligar um LED ou enviar dados de sensores para o smartphone.

A seguir está o código básico para configurar o **Bluetooth Clássico** no ESP32 e simular uma comunicação serial simples. Infelizmente, na versão atual do Wokwi.com, não poderemos simular o Bluetooth, como mostra o print da documentação do simulador Wokwi.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

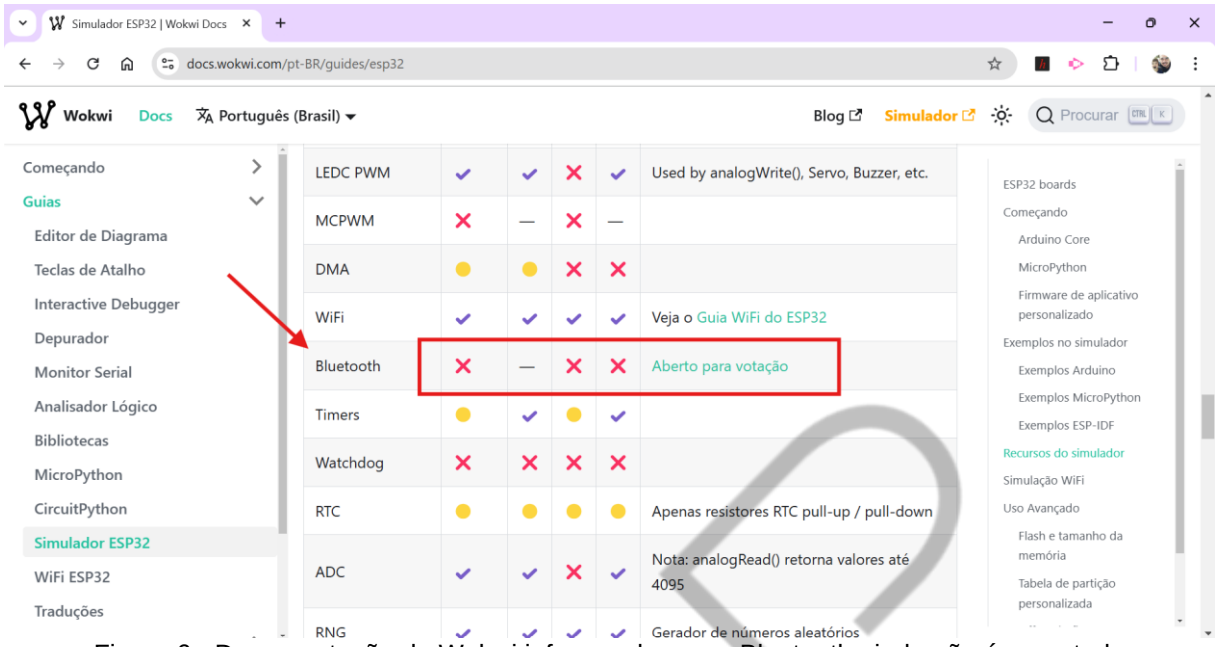


Figura 6 - Documentação do Wokwi informando que o Bluetooth ainda não é suportado
Fonte: Wokwi (2024)

Portanto, esse exemplo funciona na placa ESP32 real e não, simulada.

3.2 Exemplo de Código Bluetooth Clássico

O próximo exemplo serve para ligar e desligar um LED ligado ao GPIO 2. Se considerarmos a placa ESP32 DOIT encontrada em vários sites da Internet, esse LED é nativo na placa na cor azul, isto é, não precisa adicionar um LED ao seu projeto, pois já existe um LED soldado na placa ESP32 DOIT.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#include "BluetoothSerial.h"      // Inclui a biblioteca BluetoothSerial

BluetoothSerial SerialBT;     // Cria o objeto Serial Bluetooth

const int ledPin = 2;         // Pino do LED (GPIO 2)

void setup() {
  Serial.begin(115200); // Inicializa o Monitor Serial
  SerialBT.begin("ESP32-BT"); // Nome Bluetooth do ESP32
  pinMode(ledPin, OUTPUT); // Configura o pino do LED como saída
  digitalWrite(ledPin, LOW); // LED começa desligado
  Serial.println("Bluetooth iniciado. Aguarde conexão...");
}

void loop() {
  // Verifica se existem dados recebidos pelo Bluetooth
  if (SerialBT.available()) {
    char comando = SerialBT.read();  // Lê o caractere recebido
    Serial.print("Comando recebido: ");
    Serial.println(comando);

    // Comandos para controle do LED
    if (comando == '1') {
      digitalWrite(ledPin, HIGH);    // Liga o LED
      SerialBT.println("LED Ligado!");
    } else if (comando == '0') {
      digitalWrite(ledPin, LOW);     // Desliga o LED
      SerialBT.println("LED Desligado!");
    } else {
      SerialBT.println("Comando inválido. Envie '1' para ligar ou '0' para desligar o LED.");
    }
  }
}
```

Código-fonte 13 - Exemplo básico de uso do Bluetooth Clássico ligando e desligando um LED através do ESP32
Fonte: Elaborado pelo autor (2024)

3.2.1 Explicação do Código

1. **Configuração inicial:**
   - A biblioteca **BluetoothSerial.h** é incluída para gerenciar o Bluetooth Clássico no ESP32.
   - O método SerialBT.begin("ESP32-BT") define o nome do dispositivo Bluetooth como **"ESP32-BT"**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2. **Controle do LED:**
   - O ESP32 aguarda comandos enviados pelo Bluetooth, como os caracteres '1' ou '0'.
   - O comando '1' liga o LED, enquanto '0' o desliga.
3. **Comunicação com o usuário:**
   - Mensagens são enviadas de volta ao dispositivo conectado (smartphone ou computador) usando SerialBT.println(), confirmando a execução do comando.

3.2.2 Como Testar o Código?

1. **Carregue o código:**
   - Faça o upload do código no ESP32 utilizando a IDE Arduino.
2. **Aplicativo terminal Bluetooth:**
   - Instale um aplicativo de terminal Bluetooth em seu smartphone, como **"Serial Bluetooth Terminal"** (Android) ou **"LightBlue"** (iOS).
3. **Conexão Bluetooth:**
   - No aplicativo, procure o dispositivo **"ESP32-BT"** e conecte-se a ele.
4. **Envio de comandos:**
   - Envie o caractere **1** para ligar o LED e **0** para desligá-lo.
   - O ESP32 responderá com mensagens como **"LED Ligado!"** ou **"LED Desligado!"** no aplicativo.

3.3 Bluetooth Low Energy (BLE)

A configuração de um servidor **Bluetooth Low Energy** no ESP32 permite que ele atue como um dispositivo **peripheral**, enviando dados para dispositivos centrais, como smartphones ou outros dispositivos BLE.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O BLE é utilizado em aplicações onde o consumo de energia precisa ser reduzido, mas ainda há necessidade de comunicação, como em dispositivos vestíveis, sensores remotos e dispositivos IoT. No BLE, os dados são organizados em **serviços** e **características**, que representam os atributos do dispositivo e podem ser lidos ou escritos pelo cliente BLE.

Um **serviço** BLE é um grupo de informações relacionadas, como leituras de temperatura, e as **características** são os dados individuais dentro desse serviço, como o valor da temperatura em si.

Usando a biblioteca **BLEDevice.h**, o ESP32 pode ser configurado como um servidor BLE, criando um serviço e características que podem ser acessados por um dispositivo central. Esse dispositivo central pode ser, por exemplo, um smartphone rodando um aplicativo BLE, como o **LightBlue** ou **nRF Connect**.

No exemplo a seguir, configuramos o ESP32 como um servidor BLE básico que transmite dados para acender um LED.

3.3.1 Código Controle de LED com BLE

Novamente, até a escrita desse capítulo, o exemplo a seguir não é suportado pelo Wokwi.com. Portanto, o único jeito de rodá-lo é por meio de um ESP32 real.

Figura 7 - ESP32 modelo DOIT
Fonte: Random Nerd Tutorials (2016)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

BLEServer *pServer = NULL;
BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
bool oldDeviceConnected = false;

const int LED_PIN = 2;  // Pino do LED
bool led = false;        // Estado do LED

#define   SERVICE_UUID   "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
// UUID do serviço BLE
#define    CHARACTERISTIC_UUID_RX    "6E400002-B5A3-F393-E0A9-
E50E24DCCA9E"  // UUID para recebimento de dados

// Classe de callback para conexão Bluetooth
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) {
    deviceConnected = true;
  };

  void onDisconnect(BLEServer *pServer) {
    deviceConnected = false;
  }
};

// Classe de callback para receber dados
class MyCallbacks : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    std::string rxValue = pCharacteristic->getValue();

    if (rxValue.length() > 0) {
      if (rxValue == "led"){//Comando recebido que alterna o LED
        led = !led;  // Alterna o estado do LED
        digitalWrite(LED_PIN, led ? HIGH : LOW);

        // Responde via BLE
        String resposta = led ? "LED Ligado" : "LED Desligado";
        pCharacteristic->setValue(resposta.c_str());
        pCharacteristic->notify();

        Serial.println(resposta); // Log no Monitor Serial
      }
    }
  }
};

void setup() {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT); // Configura o pino do LED como saída
  digitalWrite(LED_PIN, LOW); // LED começa desligado

  // Inicializa o dispositivo BLE
  BLEDevice::init("ESP32-LED");

  // Configura o servidor BLE
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Cria o serviço BLE
  BLEService *pService = pServer->createService(SERVICE_UUID);

  // Cria a característica BLE para receber dados
  pCharacteristic = pService->createCharacteristic(
      CHARACTERISTIC_UUID_RX,
      BLECharacteristic::PROPERTY_WRITE                          |
BLECharacteristic::PROPERTY_NOTIFY);

  pCharacteristic->setCallbacks(new MyCallbacks());
  pCharacteristic->addDescriptor(new BLE2902());

  // Inicia o serviço e a publicidade BLE
  pService->start();
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  BLEDevice::startAdvertising();

  Serial.println("Servidor BLE ativo. Aguardando comandos...");
}

void loop() {
  // Gerencia o estado da conexão
  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();
    Serial.println("Reconectando...");
    oldDeviceConnected = deviceConnected;
  }

  if (deviceConnected && !oldDeviceConnected) {
    Serial.println("Dispositivo conectado!");
    oldDeviceConnected = deviceConnected;
  }
}
```

Código-fonte 14 - Exemplo básico de BLE para ligar/desligar um LED
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.2 Explicação do Código BLE

1. **Pino do LED:**
   - Definido no GPIO 2 (LED_PIN).
   - A variável **led** armazena o estado atual do led (ligado/desligado).
2. **Comando BLE:**
   - A característica BLE **CHARACTERISTIC_UUID_RX** recebe comandos.
   - Se o comando recebido for **"led"**, o estado do led é alternado (liga ou desliga).
   - Uma resposta é enviada de volta ao cliente BLE para confirmar a ação.
3. **Resposta ao cliente:**
   - O ESP32 responde com a mensagem **"LED Ligado"** ou **"LED Desligado"**, usando notify().
4. **Publicidade BLE:**
   - O servidor BLE anuncia o dispositivo com o nome **ESP32-LED**.
   - O cliente BLE (como um aplicativo no smartphone) pode se conectar ao ESP32 e enviar comandos.

3.3.3 Como Testar

1) Faça upload do código no ESP32 usando a **IDE Arduino**.
2) Utilize um **aplicativo BLE**, como **LightBlue** (iOS) ou **nRF Connect** (Android), para se conectar ao dispositivo chamado **ESP32-LED**.
3) Escreva o comando **"led"** na característica configurada para enviar dados.
4) Observe no Monitor Serial e no pino GPIO 2 que o **estado do farol** (LED) será alternado.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.4 Saída Esperada

- Ao enviar o comando **"led"**, o LED no GPIO 2 alterna entre ligado e desligado.
- O Monitor Serial exibirá:
  - LED Ligado;
  - LED Desligado.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

O uso do ESP32 em sistemas embarcados inteligentes e no contexto da IA representa uma poderosa convergência entre conectividade, eficiência energética e processamento em tempo real.

Neste capítulo, exploramos os principais recursos do ESP32, incluindo **interrupções**, **Wi-Fi** e **Bluetooth**, demonstrando como esses elementos são fundamentais para a construção de soluções eficazes e inovadoras.

Através de exemplos práticos, foi possível compreender a utilização de interrupções para capturar eventos críticos sem desperdício de recursos, garantindo alta responsividade em aplicações como contagem de giros e monitoramento de acessos.

Além disso, a conectividade Wi-Fi do ESP32 foi abordada como um componente central para integração com a IoT, permitindo envio e recebimento de dados em tempo real, seja por meio de redes locais ou na nuvem.

Com o uso da biblioteca **WiFi.h**, o ESP32 mostrou sua versatilidade em operar como cliente ou ponto de acesso, abrindo caminho para projetos de monitoramento remoto, automação residencial e análise de dados para modelos de IA.

Da mesma forma, o Bluetooth, tanto na versão Clássica quanto no BLE, permitiu explorar a comunicação eficiente com dispositivos móveis e sensores, criando interações robustas e de baixo consumo energético.

Essas funcionalidades tornam o ESP32 uma ferramenta essencial para projetos inteligentes, especialmente quando integrados a modelos de IA. A capacidade de capturar dados em tempo real, transmitir informações e responder de forma eficiente a eventos externos viabiliza a implementação de soluções como monitoramento de saúde, agricultura de precisão e sistemas industriais.

Em todos os exemplos, destacamos a importância do **uso otimizado de recursos**, mostrando como interrupções e conectividade podem coexistir em sistemas embarcados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por fim, a combinação do ESP32 com técnicas avançadas de IA amplia as possibilidades de inovação, ao mesmo tempo em que promove a integração entre o mundo físico e digital.

Seja em um sistema de controle de acesso, monitoramento de sensores ou aplicações críticas no ambiente hospitalar, o ESP32 se destaca por sua capacidade de **eficiência, baixo custo e flexibilidade**, tornando-se um aliado indispensável para profissionais e estudantes que buscam aplicar IA em projetos reais.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

BARRETT, Steven F.; PACK, Daniel J. **Microcontrollers fundamentals for engineers and scientists**. Morgan & Claypool Publishers, 2006.

FRIHA, Othmane *et al*. Internet of things for the future of smart agriculture: A comprehensive survey of emerging technologies. **IEEE/CAA Journal of Automatica Sinica**, v. 8, n. 4, p. 718-752, 2021.

INSTALIING the ESP32 Board in Arduino IDE (Windows, Mac OS X, Linux). **Random Nerd Tutorials**, 2016. Disponível em: <https://randomnerdtutorials.com/installing-the-esp32-board-in-arduino-ide-windows-instructions/>. Acesso em: 17 dez. 2024.

MANUAL do usuário da placa de desenvolvimento de microcontrolador JOY-iT NODEMCU ESP32. **Manuals.plus**, 02 de dezembro de 2021. Disponível em: <https://manuals.plus/pt/joy-it/nodemcu-esp32-microcontroller-development-board-manual>. Acesso em: 17 dez. 2024.

OLIVEIRA, Franklin et al**. Internet of Intelligent Things:** A convergence of embedded systems, edge computing and machine learning. Internet of Things, p. 101153, 2024.

POSCH, Maya. **Hands-On Embedded Programming with C++ 17:** Create versatile and robust embedded solutions for MCUs and RTOSes with modern C++. Packt Publishing Ltd, 2019.

SIMULADOR ESP32. **Wokwi**, 2024. Disponível em: <https://docs.wokwi.com/pt-BR/guides/esp32>. Acesso em: 17 dez. 2024.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com